

Improving Manim Video Quality with Free Llama-3.3 Models

Since you are using the free 'llama-3.3-70b-versatile' model on Groq and experiencing issues with positioning and unwanted text, here are several free and cheap methods to fix these problems without upgrading to paid services.

1. Prompt Engineering (The Most Effective Free Method)

The Llama-3.3 model is capable but needs very strict instructions to handle spatial reasoning and formatting correctly.

A. Enforce a Coordinate System

Manim uses a coordinate system where the center is (0,0), and the visible frame is roughly X: [-7, 7] and Y: [-4, 4]. You must explicitly tell the model this.

Add this to your System Prompt:

CRITICAL COORDINATE RULES:

1. The screen center is (0,0).
2. The visible X-range is [-7, 7].
3. The visible Y-range is [-4, 4].
4. To place objects, visualize a 3x3 grid.
5. PREVENT OVERLAP: Always use `.next_to(object, DIRECTION, buff=0.5)` instead of absolute coordinates when stacking items.
6. When using ``UP``, ``DOWN``, ``LEFT``, ``RIGHT``, treat them as unit vectors of length 1.

B. Negative Constraints for "Unwanted Text"

Models often like to be "chatty" or include markdown like `Here is the code:`.

Add this to your System Prompt:

OUTPUT RULES:

1. RETURN ONLY CODE.
2. DO NOT write "Here is the code", "In this scene", or any other conversational text.
3. DO NOT use markdown code blocks (````python`). Just the raw code.
4. DO NOT import anything other than ``from manim import *``.
5. IF you feel the need to explain, use Python comments ``#`` inside the code.

2. Model Configuration (Free on Groq)

You can tweak how you call the model on Groq to get better results.

A. Lower Temperature

For code generation, creativity is bad. It leads to hallucinations (random shapes/letters).

- Set **temperature** to **0.1** or **0.0**.
- Set **top_p** to **0.1**.

B. Switch to a Reasoning Model

Groq offers **DeepSeek R1 Distill Llama 70B** for free (check availability).

- This model is trained to "reason" before answering. It is excellent for spatial layout tasks.
- **Why it helps:** It will "think" (e.g., "I need to place the circle to the left of the square so they don't overlap") before generating the code.
- **Tip:** If you use this, you might need to strip the `<think>...</think>` tags from the output.

3. Robust Code Cleaning (Post-Processing)

Even with good prompts, the model might output some text. You can fix this with a simple Python function that extracts *only* the code.

Python Helper Function:

```
import re

def extract_manim_code(llm_response: str) -> str:
    # 1. Try to find code inside markdown blocks
    pattern = r"```python\n(.*?)```"
    match = re.search(pattern, llm_response, re.DOTALL)
    if match:
        return match.group(1)

    # 2. If no markdown, looks for the first import
    if "from manim import" in llm_response:
        return llm_response[llm_response.find("from manim import"):]

    return llm_response # Fallback
```

4. One-Shot or Few-Shot Examples

Providing just *one* perfect example in the prompt drastically improves performance.

Add an Example to your Prompt:

```
EXAMPLE SCENE:
# Script: "A circle appears and moves to the left."
class Scene1(Scene):
```

```
def construct(self):
    circle = Circle()
    self.play(Create(circle))
    self.play(circle.animate.to_edge(LEFT, buff=1.0))
    self.wait(1)
```

5. Iterative Refinement (Agentic Flow)

If a scene fails (video generation error), instead of showing it, feed the *error message* back to the model.

Workflow:

1. Generate Code.
2. Try to Render.
3. If specific error (e.g., `NameError: name 'Square' is not defined`), send this error back to the LLM: *"The code you generated caused this error: {error}. Fix it and return strict Python code."*
4. Retry (up to 3 times).

Summary Recommendation

- **Immediate Fix:** Update your system prompt with the "Coordinate Rules" and "Output Rules" above.
- **Quick Win:** Set `temperature=0.1` in your Groq client.
- **Next Step:** Implement the `extract_manim_code` cleaner.